Optimized Portfolios Out-of-Sample

Jerzy Pawlowski *jp3900@nyu.edu*

NYU Tandon School of Engineering

May 22, 2025

NYU | TANDON SCHOOL OF ENGINEERING

The Covariance of Stock Returns

Estimating the covariance of stock returns is complicated because their date ranges may not overlap in time. Stocks may trade over different date ranges because of IPOs and corporate events (takeovers, mergers).

The function `cov()` calculates the covariance matrix of time series. The argument `use="pairwise.complete.obs"` removes `NA` values from pairs of stock returns.

But removing `NA` values in pairs of stock returns can produce covariance matrices which are not positive semi-definite.

The reason is because the covariance are calculated over different time intervals for different pairs of stock returns.

Matrices which are not positive semi-definite may not have an inverse matrix, but they have a generalized inverse.

The function `MASS::ginv()` calculates the generalized inverse of a matrix.

```
> # Load stock returns
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_returns.RData
> datev <- zoo::index(na.omit(retstock$GOOGL))
> retp <- retstock[datev] # Subset the returns to GOOGL
> # Remove the stocks with any NA values
> numna <- sapply(retp, function(x) sum(is.na(x)))
> retp <- retp[, numna==0]
> nstocks <- NCOL(retp)
> datev <- zoo::index(retp)
> symbolv <- colnames(retp)
> retis <- retp["/2014"] # In-sample returns
> raterf <- 0.03/252
> retx <- (retis - raterf) # Excess returns
> # Calculate the covariance ignoring NA values
> covmat <- cov(retp, use="pairwise.complete.obs")
> sum(is.na(covmat))
> # Calculate the inverse of covmat
> invmat <- solve(covmat)
> round(invmat %*% covmat, digits=5)
> # Calculate the generalized inverse of covmat
> invreg <- MASS::ginv(covmat)
> all.equal(unname(invmat), invreg)
```

Generalized Inverse of Singular Covariance Matrices

The standard inverse of a positive semi-definite matrix $\mathbb{C}$ can be calculated from its *eigenvalues* $\mathbb{D}$ and its *eigenvectors* $\mathbb{O}$ as follows:

$$\mathbb{C}^{-1} = \mathbb{O}\,\mathbb{D}^{-1}\,\mathbb{O}^{T}$$

The covariance matrix may not be positive semi-definite if the number of time periods of returns (rows) is less than the number of stocks (columns).

In that case some of the higher order eigenvalues are zero, and the above covariance matrix inverse is singular.

But a non-positive semi-definite covariance matrix may still have a *generalized inverse*.

The *generalized inverse* $\mathbb{C}_g^{-1}$ is calculated by removing the zero eigenvalues, and keeping only the first n non-zero *eigenvalues*:

$$\mathbb{C}_g^{-1} = \mathbb{O}_n\,\mathbb{D}_n^{-1}\,\mathbb{O}_n^{T}$$

Where $\mathbb{D}_n$ and $\mathbb{O}_n$ are matrices with the higher order eigenvalues and eigenvectors removed.

The generalized inverse $\mathbb{C}_g^{-1}$ of the matrix $\mathbb{C}$ satisfies the equation:

$$\mathbb{C}\,\mathbb{C}_g^{-1}\mathbb{C} = \mathbb{C}$$

Which is a generalization of the standard inverse property: $\mathbb{C}^{-1}\mathbb{C} = \mathbb{1}$

```
> # Create rectangular matrix with collinear columns
> matv <- matrix(rnorm(10*8), nc=10)
> # Calculate covariance matrix
> covmat <- cov(matv)
> # Calculate inverse of covmat - error
> invmat <- solve(covmat)
> # Perform eigen decomposition
> eigend <- eigen(covmat)
> eigenvec <- eigend$vectors
> eigenval <- eigend$values
> # Set tolerance for determining zero singular values
> precv <- sqrt(.Machine$double.eps)
> # Calculate generalized inverse from the eigen decomposition
> notzero <- (eigenval > (precv*eigenval[1]))
> inveigen <- eigenvec[, notzero] %*%
+   (t(eigenvec[, notzero]) / eigenval[notzero])
> # Inverse property of inveigen isn't satisfied
> all.equal(inveigen %*% covmat, diag(NROW(covmat)))
> # Generalized inverse property of inveigen is satisfied
> all.equal(covmat %*% inveigen %*% covmat, covmat)
> # Calculate generalized inverse using MASS::ginv()
> invreg <- MASS::ginv(covmat)
> # Verify that inveigen is the same as invreg
> all.equal(inveigen, invreg)
```

Portfolio Optimization Strategy

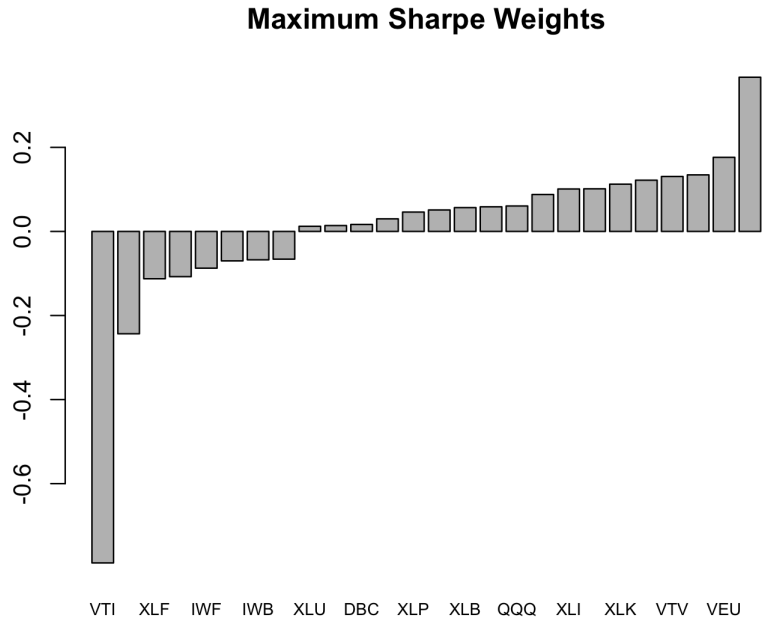
The optimal portfolio weights $\mathbf{w}$ are equal to the past in-sample excess returns $\mu = \mathbf{r} - r_f$ (in excess of the risk-free rate r_f) multiplied by the inverse of the covariance matrix $\mathbb{C}$:

$$\mathbf{w} = \mathbb{C}^{-1}\mu$$

The *portfolio optimization* strategy invests in the best performing portfolio in the past *in-sample* interval, expecting that it will continue performing well *out-of-sample*.

The *portfolio optimization* strategy consists of:

1. Calculating the maximum Sharpe ratio portfolio weights in the *in-sample* interval,
2. Applying the weights and calculating the portfolio returns in the *out-of-sample* interval.




```
> # Maximum Sharpe weights in-sample interval
> retis <- retp["/2014"]
> raterf <- 0.03/252
> retx <- (retis - raterf)
> invreg <- MASS::ginv(cov(retis, use="pairwise.complete.obs"))
> weightv <- drop(invreg %*% colMeans(retx, na.rm=TRUE))
> weightv <- weightv/sqrt(sum(weightv^2))
> names(weightv) <- colnames(retp)
```

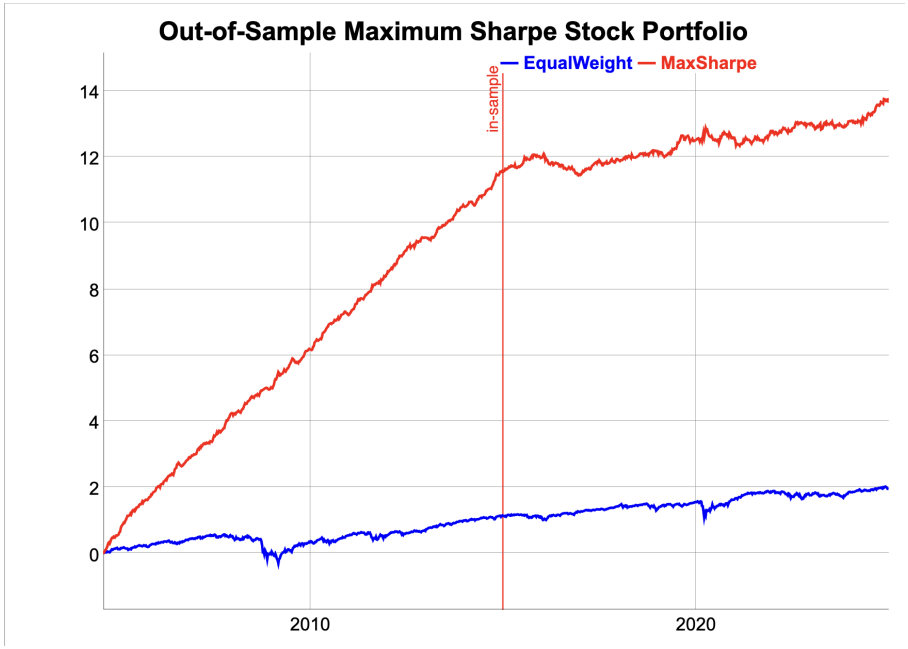
```
> # Plot portfolio weights
> barplot(sort(weightv), main="Maximum Sharpe Weights", cex.names=0
```

Optimal Stock Portfolio Out-of-Sample

The *portfolio optimization* strategy for stocks is *overfit* in the *in-sample* interval.

Therefore the strategy performance is poor in the *out-of-sample* interval.

```
> # Maximum Sharpe weights in-sample interval
> colmeanv <- colMeans(retx, na.rm=TRUE)
> covmat <- cov(retis, use="pairwise.complete.obs")
> invreg <- MASS::ginv(covmat)
> weightv <- drop(invreg %*% colmeanv)
> names(weightv) <- symbolv
> head(sort(weightv))
> tail(sort(weightv))
> # Calculate the portfolio returns
> pnls <- HighFreq::mult_mat(weightv, retp)
> pnls <- rowMeans(pnls, na.rm=TRUE)
> pnls <- xts::xts(pnls, datev)
> retew <- xts::xts(rowMeans(retp, na.rm=TRUE), datev)
> pnls <- pnls*sd(retew["/2014"])/sd(pnls["/2014"])
> wealthv <- cbind(retew, pnls)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe")
```




```
> # Calculate the in-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["/2014"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["2015/"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot of cumulative portfolio returns
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Out-of-Sample Maximum Sharpe Stock Portfolio") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(zoo::index(last(retis[, 1])), label="in-sample", strokeP
+   dyLegend(width=300)
```

Optimal Stock Weights

If an investor's utility of wealth is logarithmic: $u(w) = \log(w)$, then to maximize their utility, they should invest the fraction k_f of their wealth in a stock:

$$k_f = \frac{\bar{r}}{\sigma^2}$$

Where $\bar{r}$ is the expected stock return and σ^2 is the expected variance. The fraction k_f is called the *Kelly fraction* or the *Kelly ratio*.

The expected *logarithmic utility* u can be expanded to second order as:

$$u = \mathbb{E}[\log(1 + k_f\, r)] = \mathbb{E}[(k_f\, r - \frac{(k_f\, r)^2}{2}] =$$

$$= k_f \bar{r} - \frac{k_f^2 \sigma^2}{2}$$

The *Kelly ratio* which maximizes the utility is found by equating the derivative of utility to zero:

$$\frac{\mathrm{d}u}{\mathrm{d}k_f} = \bar{r} - k_f \sigma^2 = 0 \;\rightarrow\; k_f = \frac{\bar{r}}{\sigma^2}$$

The *Kelly criterion* favors stocks with low returns. Given two stocks with the same *Sharpe* ratios, the stock with the lower returns will have a higher *Kelly ratio*.

```
> # Objective function equal to the sum of returns
> objfun <- function(retp) sum(na.omit(retp))
> # Objective function equal to the Sharpe ratio
> objfun <- function(retp) {
+   retp <- na.omit(retp)
+   if (NROW(retp) > 12) {
+     stdev <- sd(retp)
+     if (stdev > 0) mean(retp)/stdev else 0
+   } else 0
+ }  # end objfun
> # Objective function equal to the Kelly ratio
> objfun <- function(retp) {
+   retp <- na.omit(retp)
+   if (NROW(retp) > 12) {
+     varv <- var(retp)
+     if (varv > 0) mean(retp)/varv else 0
+   } else 0
+ }  # end objfun
```

In portfolio optimization, assuming zero correlations, the *maximum Sharpe* portfolio weights are proportional to the *Kelly ratios*:

$$w_i \propto \frac{\bar{r}_i}{\sigma_i^2}$$

This is also a consequence of logarithmic utility.

Dimension Reduction of the Covariance Matrix

If the higher order singular values are very small then the inverse matrix amplifies the statistical noise in the response matrix.

The *reduced inverse* $\mathbb{C}_R^{-1}$ is calculated from the largest (lowest order) eigenvalues, up to $dimax = d$:

$$\mathbb{C}_R^{-1} = \mathbb{O}_d \, \mathbb{D}_d^{-1} \, \mathbb{O}_d^T$$

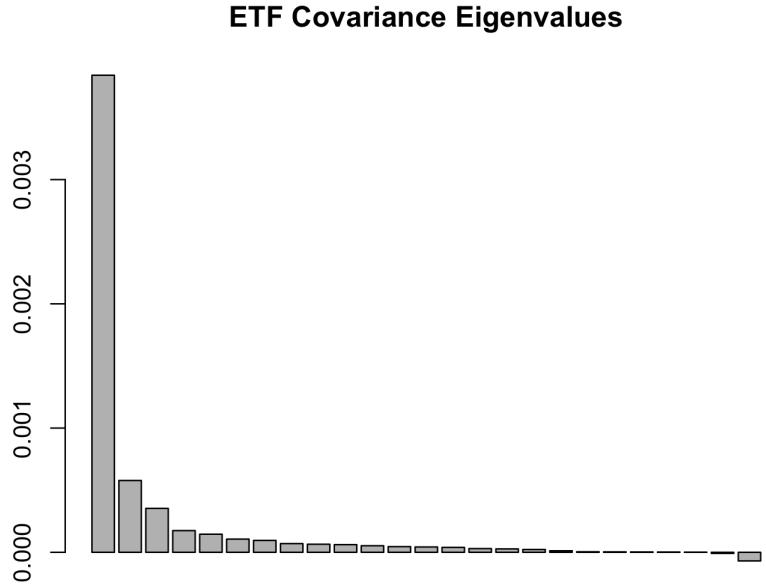
The parameter *dimax* specifies the number of eigenvalues used for calculating the *reduced inverse* of the covariance matrix of returns.

The *dimension reduction* technique calculates the *reduced inverse* of a covariance matrix by removing the very small, higher order eigenvalues, to reduce the propagation of statistical noise and improve the signal-to-noise ratio:

Even though the *reduced inverse* $\mathbb{C}_R^{-1}$ does not satisfy the matrix inverse property (so it's biased), its out-of-sample forecasts are usually more accurate than those using the exact inverse matrix.

But removing a larger number of eigenvalues increases the bias of the covariance matrix, which is an example of the *bias-variance tradeoff*.

The optimal value of the parameter *dimax* can be determined using *backtesting* (*cross-validation*).



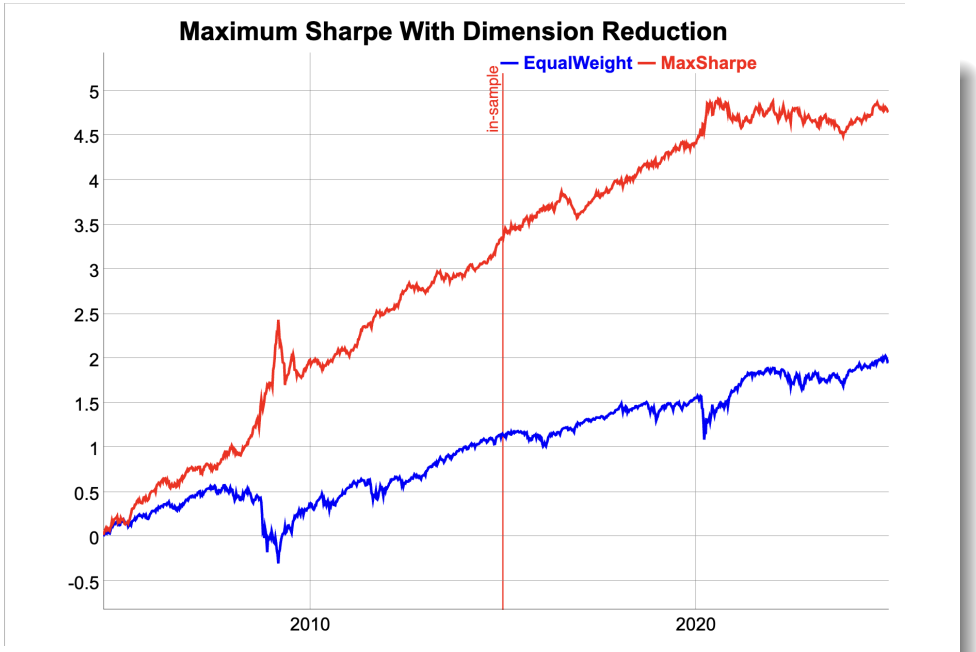

```
> # Calculate in-sample covariance matrix
> covmat <- cov(retis, use="pairwise.complete.obs")
> eigend <- eigen(covmat)
> eigenvec <- eigend$vectors
> eigenval <- eigend$values
> # Plot the eigenvalues
> barplot(eigenval, main="ETF Covariance Eigenvalues", cex.names=0.7
> # Calculate reduced inverse of covariance matrix
> dimax <- 9
> invred <- eigenvec[, 1:dimax] %*%
+   (t(eigenvec[, 1:dimax]) / eigenval[1:dimax])
> # Reduced inverse does not satisfy matrix inverse property
> all.equal(covmat %*% invred %*% covmat, covmat)
```

Maximum Sharpe Portfolio With Dimension Reduction

The *out-of-sample* performance of the *portfolio optimization* strategy can be improved by applying dimension reduction to the inverse of the covariance matrix.

The *in-sample* performance is worse because dimension reduction reduces *overfitting*.

```
> # Calculate reduced inverse of covariance matrix
> dimax <- 10
> eigend <- eigen(covmat)
> eigenvec <- eigend$vectors
> eigenval <- eigend$values
> invred <- eigenvec[, 1:dimax] %*%
+   (t(eigenvec[, 1:dimax]) / eigenval[1:dimax])
> # Calculate portfolio weights and PnLs
> colmeanv <- colMeans(retx["/2014"], na.rm=TRUE)
> weightv <- invred %*% colmeanv
> pnls <- HighFreq::mult_mat(weightv, retp)
> pnls <- rowMeans(pnls, na.rm=TRUE)
> pnls <- xts::xts(pnls, datev)
> pnls <- pnls*sd(retew["/2014"])/sd(pnls["/2014"])
```




```
> # Combine with equal weight
> wealthv <- cbind(retew, pnls)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe")
> # Calculate the in-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["/2014"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["2015/"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot of cumulative portfolio returns
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Maximum Sharpe With Dimension Reduction") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(zoo::index(last(retis[, 1])), label="in-sample", stroke
+   dyLegend(width=300)
```

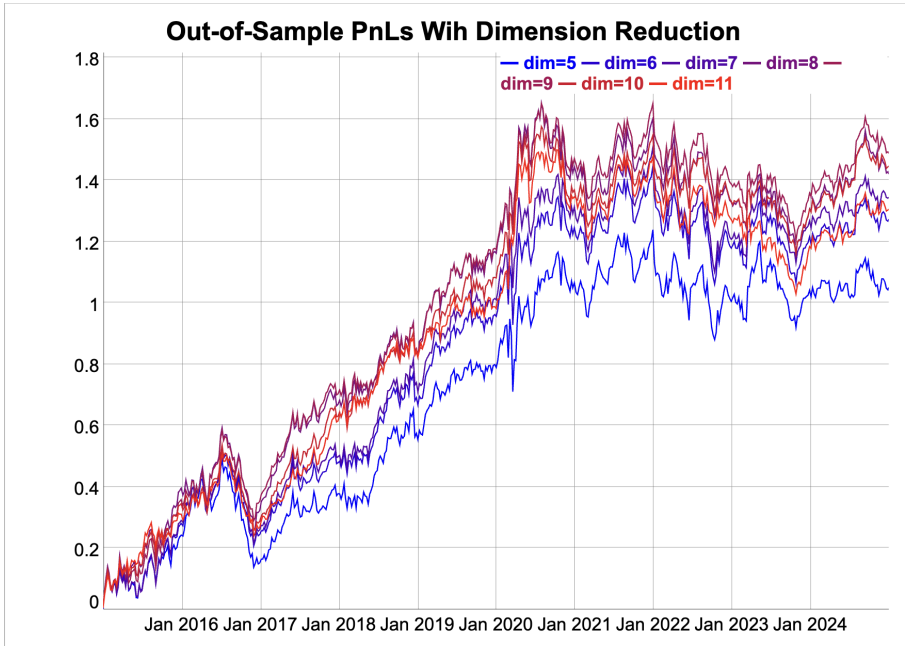
Optimal Dimension Reduction Out-of-Sample

The optimal out-of-sample dimension reduction parameter is `dimax=9`, which represents relatively weak dimension reduction.

The dependence of the out-of-sample strategy pnls on the *dimax* parameter reflects the *bias-variance tradeoff*.

If the *dimax* parameter is too small, it creates excessive bias, but if it's too large, it doesn't suppress the variance enough.

```
> # Perform loop over vector of dimension reduction parameters
> dimv <- seq(from=5, to=11)
> pnls <- mclapply(dimv, function(dimax) {
+   invred <- eigenvec[, 1:dimax] %*%
+     (t(eigenvec[, 1:dimax]) / eigenval[1:dimax])
+   # Calculate portfolio weights and PnLs
+   weightv <- invred %*% colmeanv
+   pnls <- HighFreq::mult_mat(weightv, retp)
+   pnls <- rowMeans(pnls, na.rm=TRUE)
+   pnls <- xts::xts(pnls, datev)
+   pnls*sd(retew["/2014"])/sd(pnls["/2014"])
+ }, mc.cores=ncores)  # end mclapply
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("dim=", dimv)
> profilev <- sapply(pnls["2015/"], sum)
> dimax <- dimv[which.max(profilev)]
```




```
> # Plot of cumulative portfolio returns
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> endw <- rutils::calc_endpoints(pnls["2015/"], interval="weeks")
> dygraphs::dygraph(cumsum(pnls["2015/"])[endw],
+   main="Out-of-Sample PnLs Wih Dimension Reduction") %>%
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=300)
```

Maximum Sharpe Portfolio With Return Shrinkage

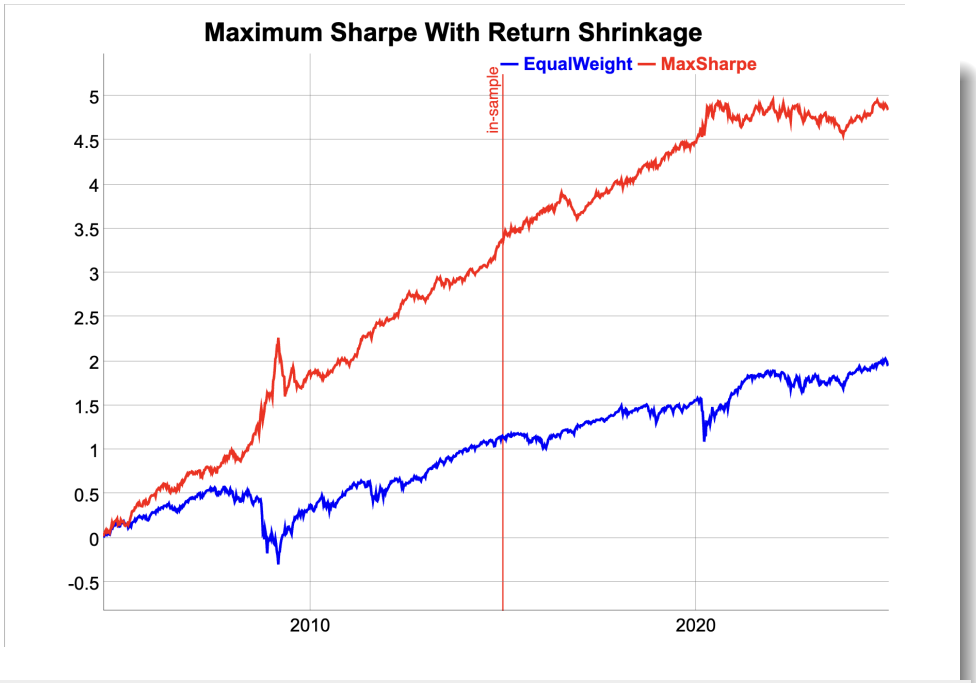
To further reduce the statistical noise, the individual returns r_i can be *shrunk* to the average portfolio returns $\bar{r}$:

$$r_i' = (1 - \alpha)\, r_i + \alpha\, \bar{r}$$

The parameter α is the *shrinkage* intensity, and it determines the strength of the *shrinkage* of individual returns to their mean.

If $\alpha = 0$ then there is no *shrinkage*, while if $\alpha = 1$ then all the returns are *shrunk* to their common mean: $r_i = \bar{r}$.

The optimal value of the *shrinkage* intensity α can be determined using *backtesting* (*cross-validation*).




```
> # Shrink the in-sample returns to their mean
> alphac <- 0.3
> retxm <- rowMeans(retx, na.rm=TRUE)
> retxis <- (1-alphac)*retx + alphac*retxm
> # Calculate portfolio weights and PnLs
> weightv <- drop(invred %*% colMeans(retxis, na.rm=TRUE))
> names(weightv) <- symbolv
> pnls <- HighFreq::mult_mat(weightv, retp)
> pnls <- rowMeans(pnls, na.rm=TRUE)
> pnls <- xts::xts(pnls, datev)
> pnls <- pnls*sd(retew["/2014"])/sd(pnls["/2014"])
```

```
> # Combine with equal weight
> wealthv <- cbind(retew, pnls)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe")
> # Calculate the in-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["/2014"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["2015/"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot of cumulative portfolio returns
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Maximum Sharpe With Return Shrinkage") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(zoo::index(last(retis[, 1])), label="in-sample", stroke
+   dyLegend(width=300)
```

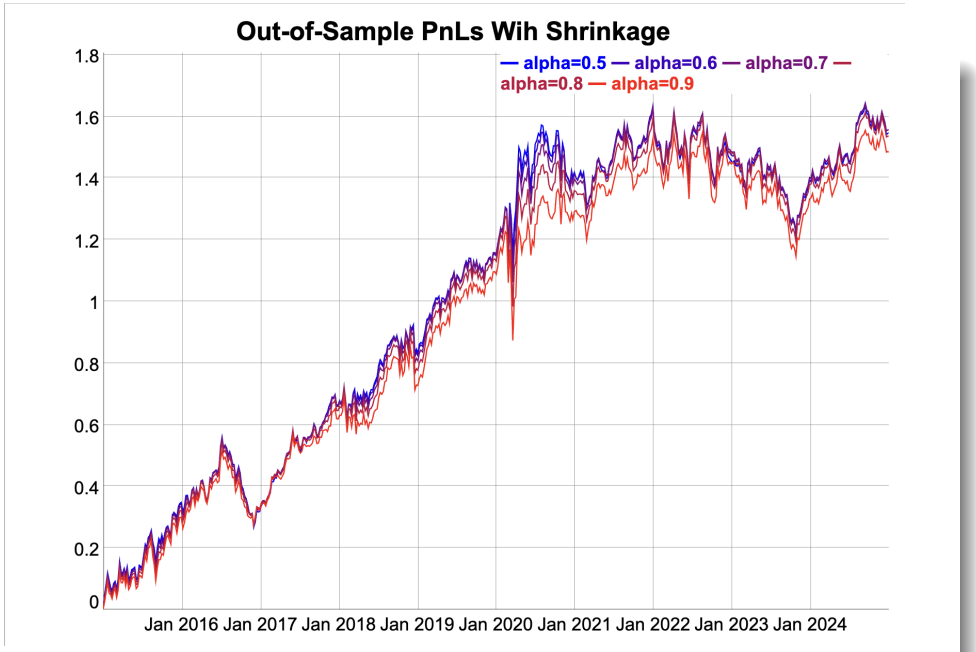
Optimal Shrinkage Parameter Out-of-Sample

The optimal out-of-sample return shrinkage parameter is equal to $\alpha = 0.6$, which represents moderate shrinkage.

The dependence of the out-of-sample strategy pnls on the α parameter reflects the *bias-variance tradeoff*.

If the α parameter is too large, it creates excessive bias, but if it's too small, it doesn't suppress the variance enough.

```
> # Perform loop over vector of shrinkage intensities
> alphav <- seq(from=0.5, to=0.9, by=0.1)
> pnls <- mclapply(alphav, function(alphac) {
+   retxis <- (1-alphac)*retx + alphac*retxm
+   weightv <- drop(invred %*% colMeans(retxis, na.rm=TRUE))
+   pnls <- HighFreq::mult_mat(weightv, retp)
+   pnls <- rowMeans(pnls, na.rm=TRUE)
+   pnls <- xts::xts(pnls, datev)
+   pnls*sd(retew["/2014"])/sd(pnls["/2014"])
+ }, mc.cores=ncores)  # end mclapply
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("alpha=", alphav)
> profilev <- sapply(pnls["2015/"], sum)
> alphac <- alphav[which.max(profilev)]
```




```
> # Plot of cumulative portfolio returns
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> endw <- rutils::calc_endpoints(pnls["2015/"], interval="weeks")
> dygraphs::dygraph(cumsum(pnls["2015/"])[endw],
+   main="Out-of-Sample PnLs Wih Shrinkage") %>%
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=300)
```

Rolling Maximum Sharpe Stock Portfolio Strategy

A *rolling portfolio optimization* strategy consists of rebalancing a portfolio over the end points:

1. Calculate the maximum Sharpe ratio portfolio weights at each end point,
2. Apply the weights in the next interval and calculate the out-of-sample portfolio returns.

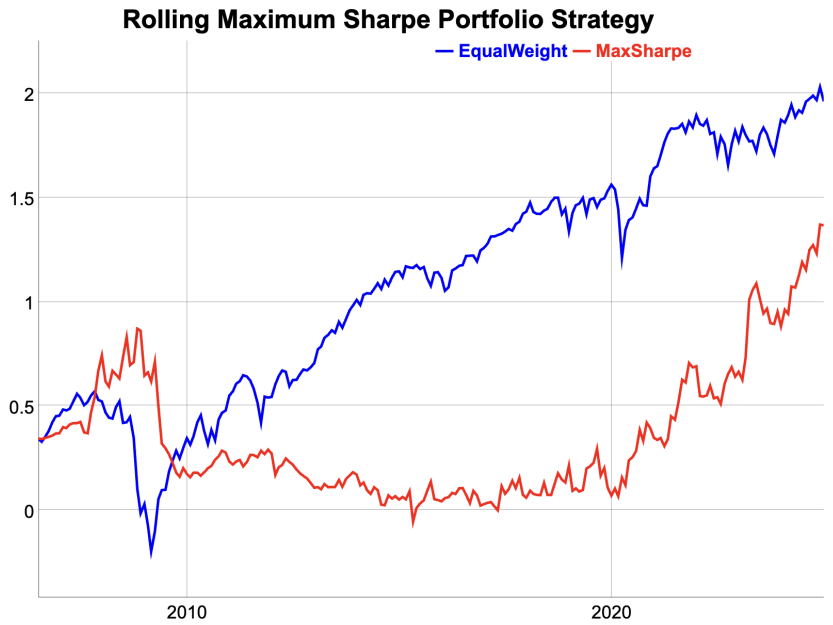
The strategy parameters are: the rebalancing frequency (annual, monthly, etc.), and the length of look-back interval.

```
> # Define monthly end points
> endd <- rutils::calc_endpoints(retp, interval="months")
> endd <- endd[endd > (nstocks+1)]
> npts <- NROW(endd) ; lookb <- 12
> startp <- c(rep_len(0, lookb), endd[1:(npts-lookb)])
> # !!! Perform parallel loop over end points - takes very long!!!
> library(parallel)  # Load package parallel
> ncores <- detectCores() - 1
> pnls <- mclapply(1:(npts-1), function(tday) {
+     # Subset the excess returns
+     retis <- retp[startp[tday]:endd[tday], ]
+     retis[is.na(retis)] <- 0
+     invreg <- MASS::ginv(cov(retis, use="pairwise.complete.obs"))
+     # Calculate the maximum Sharpe ratio portfolio weights
+     retm <- colMeans(retis, na.rm=TRUE)
+     weightv <- invreg %*% retm
+     # Zero weights if sparse data
+     zerov <- sapply(retis, function(x) (sum(x == 0) > 5))
+     weightv[zerov] <- 0
+     # Calculate in-sample portfolio returns
+     pnlis <- (retis %*% weightv)
+     # Scale the weights to the in-sample volatility
+     weightv <- weightv*sd(retm)/sd(pnlis)
+     # Calculate the out-of-sample portfolio returns
+     retos <- retp[(endd[tday]+1):endd[tday+1], ]
+     pnls <- HighFreq::mult_mat(weightv, retos)
+     pnls <- rowMeans(pnls, na.rm=TRUE)
+     xts::xts(pnls, zoo::index(retos))
+ }, mc.cores=ncores)  # end mclapply
> pnls <- rutils::do_call(rbind, pnls)
> pnls <- rbind(retew[paste0("/", start(pnls)-1)], pnls*sd(retew)/sd
```

Performance of Rolling Maximum Sharpe Strategy

The performance of the *rolling portfolio optimization* strategy can be improved by applying dimension reduction and return shrinkage.

```
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```




```
> # Plot cumulative strategy returns
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling Maximum Sharpe Portfolio Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Fast Covariance Matrix Inverse Using *RcppArmadillo*

RcppArmadillo can be used to quickly calculate the reduced inverse of a covariance matrix.

```
> # Create random matrix of returns
> matv <- matrix(rnorm(300), nc=5)
> # Reduced inverse of covariance matrix
> dimax <- 3
> eigend <- eigen(covmat)
> invred <- eigend$vectors[, 1:dimax] %*%
+   (t(eigend$vectors[, 1:dimax]) / eigend$values[1:dimax])
> # Reduced inverse using RcppArmadillo
> invarma <- HighFreq::calc_inv(covmat, dimax)
> all.equal(invred, invarma)
> # Microbenchmark RcppArmadillo code
> library(microbenchmark)
> summary(microbenchmark(
+   rcode={eigend <- eigen(covmat)
+     eigend$vectors[, 1:dimax] %*%
+ (t(eigend$vectors[, 1:dimax]) / eigend$values[1:dimax])
+   },
+   rcpp=calc_inv(covmat, dimax),
+   times=10))[, c(1, 4, 5)]  # end microbenchmark summary
```

```
arma::mat calc_inv(const arma::mat& matv,
                   arma::uword dimax = 0, // Max number
                   double eigen_thresh = 0.01) { // Thre

  if (dimax == 0) {
    // Calculate the inverse using arma::pinv()
    return arma::pinv(tseries, eigen_thresh);
  } else {
    // Calculate the reduced inverse using SVD decomposi

    // Allocate SVD
    arma::vec svdval;
    arma::mat svdu, svdv;

    // Calculate the SVD
    arma::svd(svdu, svdval, svdv, tseries);

    // Subset the SVD
    dimax = dimax - 1;
    // For no regularization: dimax = tseries.n_cols
    svdu = svdu.cols(0, dimax);
    svdv = svdv.cols(0, dimax);
    svdval = svdval.subvec(0, dimax);

    // Calculate the inverse from the SVD
    return svdv*arma::diagmat(1/svdval)*svdu.t();

  }  // end if

}  // end calc_inv
```

Portfolio Optimization Using *RcppArmadillo*

Fast portfolio optimization using matrix algebra can be implemented using *RcppArmadillo*.

```
arma::vec calc_weights(const arma::mat& returns, // Asset returns
                       Rcpp::List controll) { // List of portfolio optimization parameters

  // Unpack the control list of portfolio optimization parameters
  // Type of portfolio optimization model
  std::string method = Rcpp::as<std::string>(controll["method"]);
  // Threshold level for discarding small singular values
  double eigen_thresh = Rcpp::as<double>(controll["eigen_thresh"]);
  // Dimension reduction
  arma::uword dimax = Rcpp::as<int>(controll["dimax"]);
  // Confidence level for calculating the quantiles of returns
  double confl = Rcpp::as<double>(controll["confl"]);
  // Shrinkage intensity of returns
  double alpha = Rcpp::as<double>(controll["alpha"]);
  // Should the weights be ranked?
  bool rankw = Rcpp::as<int>(controll["rankw"]);
  // Should the weights be centered?
  bool centerw = Rcpp::as<int>(controll["centerw"]);
  // Method for scaling the weights
  std::string scalew = Rcpp::as<std::string>(controll["scalew"]);
  // Volatility target for scaling the weights
  double vol_target = Rcpp::as<double>(controll["vol_target"]);

  // Initialize the variables
  arma::uword ncols = returns.n_cols;
  arma::vec weightv(ncols, fill::zeros);
  // If no regularization then set dimax to ncols
  if (dimax == 0)  dimax = ncols;
  // Calculate the covariance matrix
  arma::mat covmat = calc_covar(returns);

  // Apply different calculation methods for the weights
  switch(calc_method(method)) {
  case methodenum::maxsharpe: {
    // Mean returns of columns
```

Strategy Backtesting Using *RcppArmadillo*

Fast backtesting of strategies can be implemented using *RcppArmadillo*.

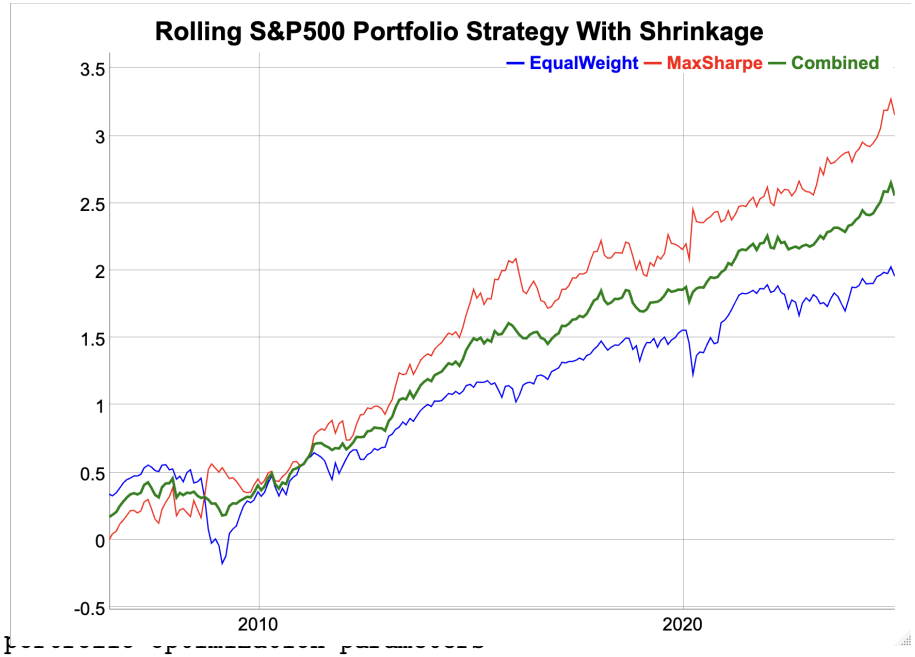
```
arma::mat roll_portf(const arma::mat& retx, // Asset excess returns
                     const arma::mat& retp, // Asset returns
                     Rcpp::List controll, // List of portfolio optimization model parameters
                     arma::uvec startp, // Start points
                     arma::uvec endp, // End points
                     double lambdaf = 0.0, // Decay factor for averaging the portfolio weights
                     double coeff = 1.0, // Multiplier of strategy returns
                     double bidask = 0.0) { // The bid-ask spread

  double lambda1 = 1-lambdaf;
  arma::uword nweights = retp.n_cols;
  arma::vec weightv(nweights, fill::zeros);
  arma::vec weights_past = arma::ones(nweights)/std::sqrt(nweights);
  arma::mat pnls = arma::zeros(retp.n_rows, 1);

  // Perform loop over the end points
  for (arma::uword it = 1; it < endp.size(); it++) {
    // cout << "it: " << it << endl;
    // Calculate the portfolio weights
    weightv = coeff*calc_weights(retx.rows(startp(it-1), endp(it-1)), controll);
    // Calculate the weights as the weighted sum with past weights
    weightv = lambda1*weightv + lambdaf*weights_past;
    // Calculate out-of-sample returns
    pnls.rows(endp(it-1)+1, endp(it)) = retp.rows(endp(it-1)+1, endp(it))*weightv;
    // Add transaction costs
    pnls.row(endp(it-1)+1) -= bidask*sum(abs(weightv - weights_past))/2;
    // Copy the weights
    weights_past = weightv;
  }  // end for

  // Return the strategy pnls
  return pnls;

}  // end roll_portf
```

Rolling Portfolio Strategy With Dimension Reduction and Shrinkage

The performance of the *rolling portfolio optimization* strategy can be improved by applying dimension reduction and return shrinkage.

The *rolling portfolio* strategy has a higher *Sharpe* ratio and also higher absolute returns than the equal weight portfolio.

The backtest simulation can be performed very quickly using C++ code and package *RcppArmadillo*.




```
> # Shift the end points to C++ convention
> endd <- (endd - 1)
> endd[endd < 0] <- 0
> startp <- (startp - 1)
> startp[startp < 0] <- 0
> # Specify dimension reduction and return shrinkage using list of
> dimax <- 9
> alphac <- 0.8
> controll <- HighFreq::param_portf(method="maxsharpe",
+   dimax=dimax, alpha=alphac)
> # Perform backtest in Rcpp - takes very long!!!
> retx <- (retp - raterf)
> pnls <- HighFreq::roll_portf(retx=retx, retp=retp,
+   startp=startp, endd=endd, controll=controll)
> pnls <- pnls*sd(retew)/sd(pnls)
```

```
> # Calculate the out-of-sample Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls, (pnls + retew)/2)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe", "Combined")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot cumulative strategy returns
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling S&P500 Portfolio Strategy With Shrinkage") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", label="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Optimal Dimension Reduction Parameter

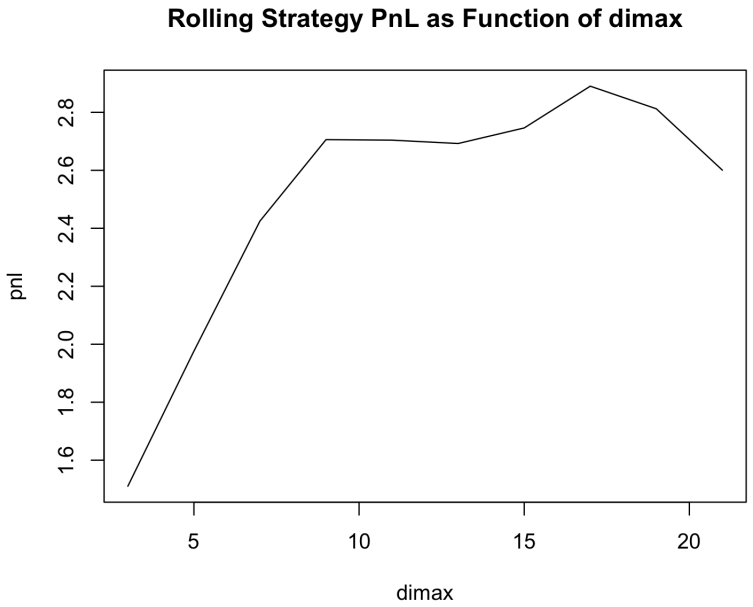
The optimal value of the dimension reduction parameter *dimax* can be determined using *backtesting*.

The optimal dimension reduction parameter for this portfolio of stocks is equal to `dimax=9`, which represents relatively weak dimension reduction.

The dependence of the out-of-sample strategy pnls on the *dimax* parameter reflects the *bias-variance tradeoff*.

If the *dimax* parameter is too small, it creates excessive bias, but if it's too large, it doesn't suppress the variance enough.

```
> # Perform backtest over vector of dimension reduction parameters
> dimv <- seq(from=3, to=21, by=2)
> pnls <- mclapply(dimv, function(dimax) {
+   controll <- HighFreq::param_portf(method="maxsharpe",
+     dimax=dimax, alpha=alphac)
+   HighFreq::roll_portf(retx=retx, retp=retp,
+     startp=startp, endd=endd, controll=controll)
+ }, mc.cores=ncores)  # end mclapply
> profilev <- sapply(pnls, sum)
> dimax <- dimv[which.max(profilev)]
```




```
> # Plot of rolling strategy PnL as a function of dimax
> plot(x=dimv, y=profilev, t="l", xlab="dimax", ylab="pnl",
+   main="Rolling Strategy PnL as Function of dimax")
```

Optimal Shrinkage Parameter

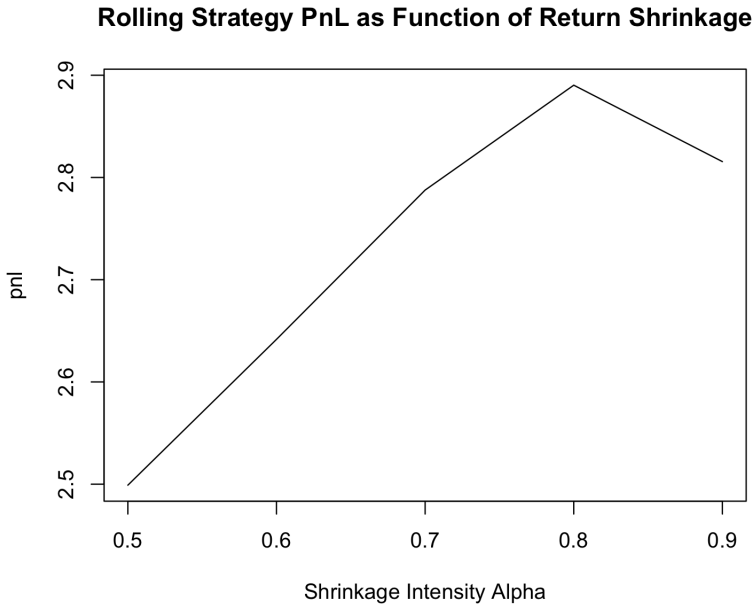
The optimal value of the return shrinkage intensity parameter α can be determined using *backtesting*.

The best return shrinkage parameter for this portfolio of stocks is equal to $\alpha = 0.8$, which represents strong shrinkage.

The dependence of the out-of-sample strategy pnls on the α parameter reflects the *bias-variance tradeoff*.

If the α parameter is too large, it creates excessive bias, but if it's too small, it doesn't suppress the variance enough.

```
> # Perform backtest over vector of shrinkage intensities
> alphav <- seq(from=0.5, to=0.9, by=0.1)
> pnls <- mclapply(alphav, function(alphac) {
+   controll <- HighFreq::param_portf(method="maxsharpe",
+       dimax=dimax, alpha=alphac)
+   HighFreq::roll_portf(retx=retx, retp=retp,
+       startp=startp, endd=endd, controll=controll)
+ }, mc.cores=ncores)  # end mclapply
> profilev <- sapply(pnls, sum)
> alphac <- alphav[which.max(profilev)]
```




```
> # Plot of rolling strategy PnL as a function of shrinkage intensit
> plot(x=alphav, y=profilev, t="l",
+   main="Rolling Strategy PnL as Function of Return Shrinkage",
+   xlab="Shrinkage Intensity Alpha", ylab="pnl")
```

Optimal Look-back Interval

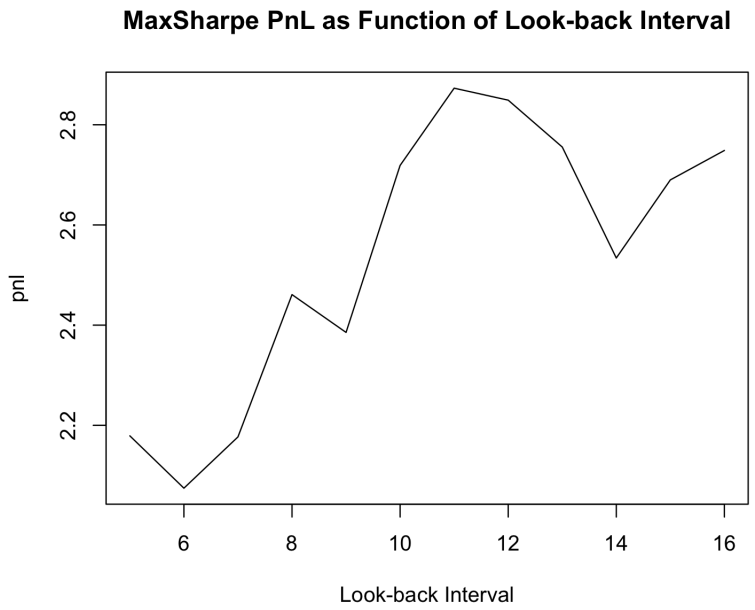
The optimal length of the look-back interval can be determined using *backtesting*.

The optimal length of the look-back interval for this portfolio of stocks is equal to `lookb=10` months, which roughly agrees with the research literature on momentum strategies.

The dependence on the length of the *look-back interval* is an example of the *bias-variance tradeoff*.

If the *look-back interval* is too long, then the data has large *bias* because the distant past may have little relevance to today.

But if the *look-back interval* is too short, then there's not enough data, and estimates will have high *variance*.



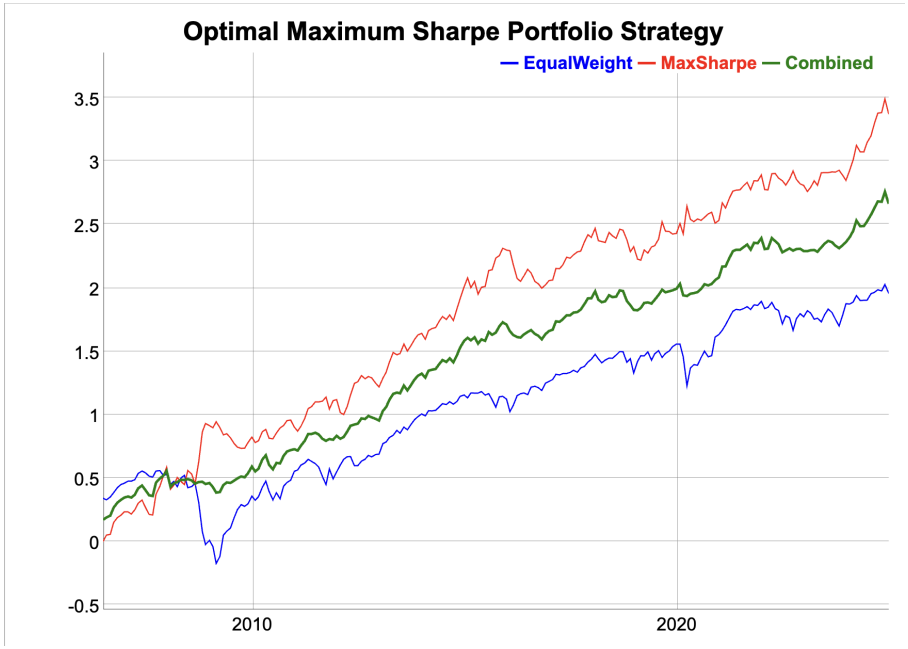

```
> # Create list of model parameters
> controll <- HighFreq::param_portf(method="maxsharpe",
+       dimax=dimax, alpha=alphac)
> # Perform backtest over look-backs
> lookbv <- seq(from=5, to=16, by=1)
> pnls <- mclapply(lookbv, function(lookb) {
+   startp <- c(rep_len(0, lookb), endd[1:(npts-lookb)])
+   startp <- (startp - 1) ; startp[startp < 0] <- 0
+   HighFreq::roll_portf(retx=retx, retp=retp,
+     startp=startp, endd=endd, controll=controll)
+ }, mc.cores=ncores)  # end mclapply
> profilev <- sapply(pnls, sum)
> lookb <- lookbv[which.max(profilev)]
```

```
> # Plot of rolling strategy PnL as a function of look-back interval
> plot(x=lookbv, y=profilev, t="l", main="MaxSharpe PnL as Function
+   xlab="Look-back Interval", ylab="pnl")
```

Optimal Portfolio Optimization Strategy

The optimal *rolling portfolio optimization* strategy, with dimension reduction and return shrinkage, has both a higher *Sharpe* ratio and higher absolute returns than the equal weight portfolio.

Combining the optimal rolling portfolio strategy with the equal weight portfolio results in an even higher *Sharpe* ratio.



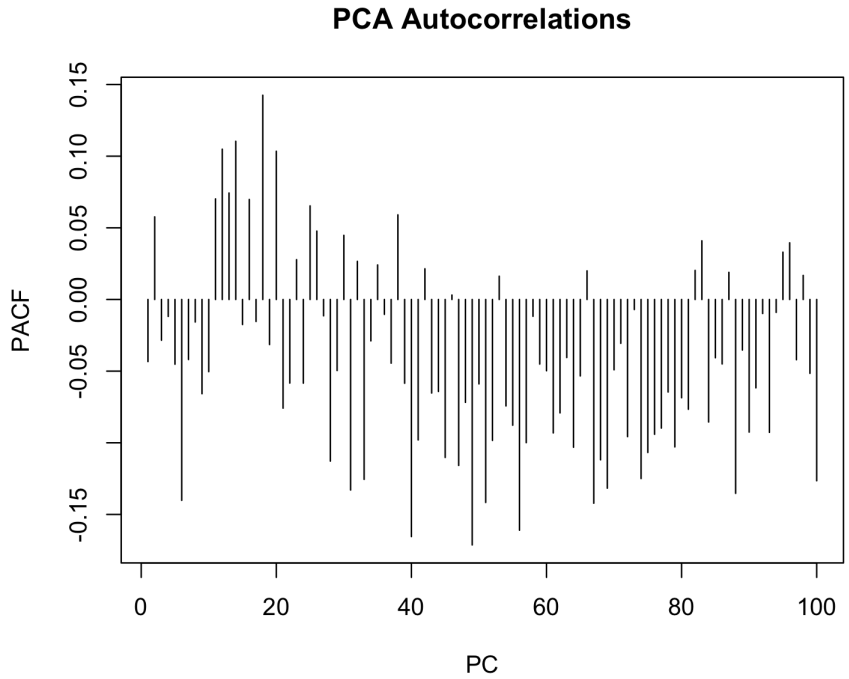

```
> # Calculate the out-of-sample Sharpe and Sortino ratios
> pnls <- pnls[[whichmax]]
> pnls <- pnls*sd(retew)/sd(pnls)
> wealthv <- cbind(retew, pnls, (pnls + retew)/2)
> colnames(wealthv) <- c("EqualWeight", "MaxSharpe", "Combined")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Dygraph the cumulative wealth
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimal Maximum Sharpe Portfolio Strategy") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", label="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Autocorrelations of PCA Returns

The *Principal Component* portfolios (*PC*s) have either trending or mean-reverting characteristics.

The lower order *PCs* exhibit greater trending (positive autocorrelations) than individual stocks.

The higher order *PCs* exhibit greater mean reversion (negative autocorrelations) than the lower order ones.




```
> # Calculate the autocorrelations of the PCA time series
> pacv <- apply(retpca[, 1:100], 2, function(x)
+   sum(pacf(x, lag=10, plot=FALSE)$acf))
> plot(pacv, type="h", main="PCA Autocorrelations",
+      xlab="PC", ylab="PACF")
```

Momentum Strategy for PCA Portfolios

The momentum strategy performs better for *PCA* portfolios than for individual stocks for two reasons:

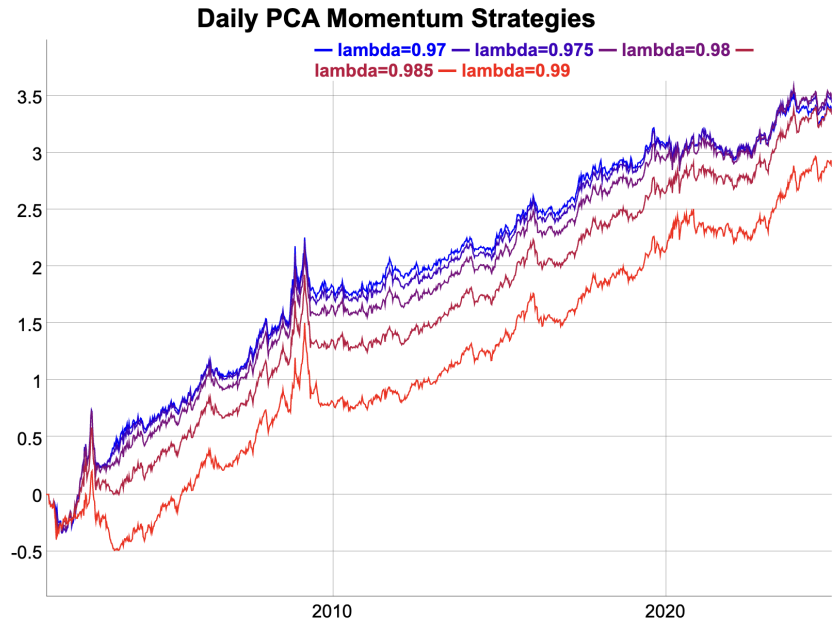
- The *PCA* returns are uncorrelated to each other,
- The lower order *PCA* returns have more positive autocorrelations than individual stocks.
- The higher order *PCA* returns have more negative autocorrelations than individual stocks.

If the stock returns are uncorrelated then the weights of the *maximum Sharpe* portfolio are proportional to the *Kelly ratios* (the returns divided by their variance):

$$w_i = \mathbb{C}^{-1}\bar{r} = \frac{\bar{r}_i}{\sigma_i^2}$$

Where $\mathbb{C}$ is the covariance matrix of returns (which is diagonal in this case).

If the momentum weights are chosen to be the *Kelly ratios*, then the momentum strategy is equivalent to the *maximum Sharpe* optimal portfolio strategy.




```
> # Simulate daily PCA momentum strategies for multiple lambdaf pa
> dimax <- 30
> lambdav <- seq(0.97, 0.99, 0.005)
> pnls <- mclapply(lambdav, btmomdailyhold, retp=retpca[, 1:dimax]
> pnls <- lapply(pnls, function(pnl) volew*pnl/sd(pnl))
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("lambda=", lambdav)
> pnls <- xts::xts(pnls, datev)
```

```
> # Plot Sharpe ratios of momentum strategies
> sharper <- sqrt(252)*sapply(pnls, function(pnl) mean(pnl)/sd(pnl))
> plot(x=lambdav, y=sharper, t="l",
+   main="PCA Momentum Sharpe as Function of Decay Factor",
+   xlab="lambdaf", ylab="Sharpe")
> # Plot dygraph of daily PCA momentum strategies
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> endw <- rutils::calc_endpoints(pnls, interval="weeks")
> dygraphs::dygraph(cumsum(pnls)[endw],
+   main="Daily PCA Momentum Strategies") %>%
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=400)
```

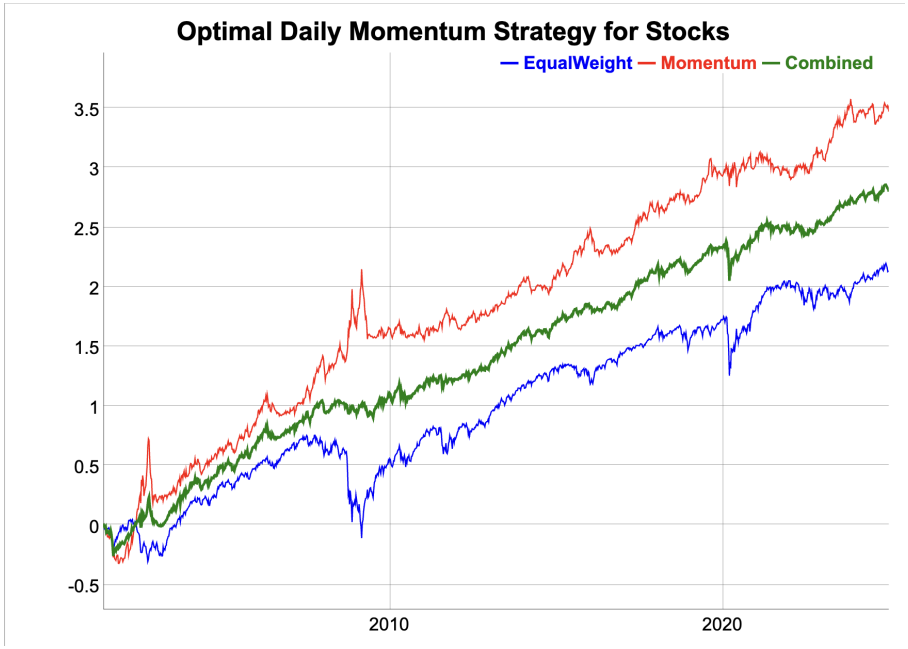
Optimal PCA Momentum Strategy

The *PCA* momentum strategy using only the lowest order *PCs* performs well when combined with the index.

But this is thanks to using the in-sample *PCs*.

The best performing *PCA* momentum strategy has a relatively small decay factor λ, so it's able to quickly adjust to changes in market direction.

```
> # Calculate best pnls of PCA momentum strategy
> whichmax <- which.max(sharper)
> lambdav[whichmax]
> pnls <- pnls[, whichmax]
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls, 0.5*(retew + pnls))
> colnames(wealthv) <- c("EqualWeight", "Momentum", "Combined")
> cor(wealthv)
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```




```
> # Plot dygraph of stock index and PCA momentum strategy
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimal Daily Momentum Strategy for Stocks") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```